

基础16 Array new/delete 与 std::allocator 的引入

① 基础出用法 `int *a = new int[3]{1, 42, 23};`
`delete []a;` 连续分配

`new []` 必须对应 `delete []` 吗

对上面的代码 `delete` 后的 `[]` 可有可无, 因为在 `new` 时就已经指定了分配内存的多少, 而 `delete` 后也确实未指定具体的个数, 因此当我们 `new` 时, 系统在堆上分配的内存比我们想要的要更多

② 不加 `[]` 会导致内存泄漏的情况 (为什么)

```
class Test {  
public: Test(int a) { _ptr = new int(a); }  
    ~Test() { delete _ptr; }  
private: int *_ptr; }  
Test *t = new Test[3]{Test(1), Test(2),  
                      Test(3)};
```

`delete t;` 只调用了一次析构, 并且还抛出异常

③ 重载 `operator new/delete` 有什么弊端

每一个类都要实现一遍, 对于软件工程来说实在不合理
为解决代码重复 `std::allocator` 出现了